Shorter K215 Proofs through Smaller Interfaces

A pinned review, checked AX refactors, a Rocq command adapter, and a direct route from the seed property to Infinity

Research proposal for the ProveIt repository — October 21, 2026

Reviewed revision. `6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95`. The checkout was obtained from the GitHub repository and pinned before experimentation. All comparisons in this report refer to that revision, rather than a moving branch or a historical proposal copy.

Recommendation. Adopt the small AX certificate refactor first: obtain successor addresses from the literal program instead of storing them a second time. For the maintained pair of dominated/permuted AX developments, share the semantic proof as well: their only operation-order difference is a commuting pair with disjoint register footprints. Bring Rocq's enumerator to the command/codec interface already used by Lean. The larger mathematical opportunity is to derive ordinary Infinity directly from the seed property, bypassing the singleton-index construction on the K215 lower route.

Change	Concrete result	Validation boundary
Local AX annotation erasure	K215: 7,563 to 6,533 bytes; no new project module	Targeted Lean check with unchanged root/seed clients
Shared AX alternative	322 to 226 lines across both lanes	One semantic evaluation, two literal bindings, both root clients
Rocq command/codec interface	Exact factorization on every natural-valued state	Eleven-module compatibility build; recursive `coqchk`
Direct Infinity route	Ordinary Infinity in the actual Lean quotient	40-module fresh check; independent Rocq core; endpoint integration remains separate

The two AX patches are *alternatives*, not additive savings. The Rocq adapter is an additive pilot, not a claimed net source reduction. No new complete K215 endpoint build is claimed for this packet. The source audit and compiler receipts in the archive distinguish the checks performed here from the repository's existing endpoint evidence.

1 What is already proved, and what is actually large

The maintained Lean endpoint states

$$\mathrm{Halt}(K_{215}, \mathrm{blank}) \quad \longleftrightarrow \quad \mathrm{BProv}(\mathrm{ZFCax}_s, [\,], \bot).$$

Negating both sides gives nonhalting if and only if syntactic consistency. The corresponding Rocq endpoint concerns the same literal two-symbol, 215-working-state machine. The dense table has 430 actions and includes its initialization in the working-state count. Its SHA-256 is

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`.

These statements and their existing proof receipts are recorded in the repository's K215 verification directory [1, 3].

An important distinction survives every simplification proposed here. The existing results are external Lean/Rocq theorems about a syntactic ZFC proof predicate. The repository explicitly says that it has not supplied an internal PA proof or a derivation encoded in ZFC's own syntax [1]. A refactor should preserve the exact currently checked theorem; it should not silently promote that theorem to the separate internal-formalization milestone.

1.1 An independently reproduced current census

The supplied read-only audit follows project imports from `BusyBeaver.ZFC.KEndpoint215`. It finds **178 project modules**, **2,093,112 source bytes**, and **40,517 physical lines**. The only external import it encounters is the shipped Lean `Init` boundary. The module count and byte total exactly reproduce phase fourteen's published census [2]. Physical lines include comments and literal data; they are not a count of logical inferences or a measure of proof-term size.

Current source	Physical lines	Bytes
`KEndpoint215.lean`	54	2,653
`CorrectDense215.v`	18	718
`EnumPerm.lean`	545	22,788
`L2AxiomsPerm.lean`	161	7,563
`L2AxiomsDom.lean`	161	7,536
`KSource215Data.lean`	1,575	239,254
`KSource215Program.lean`	200	129,741
`InductiveAttempts.lean`	397	18,624

The final compositions are already short. Most prospective savings must come from repeated semantics, administrative certificate data, or a mathematical detour in the imported development. Compressing the final biconditional is therefore a low priority.

1.2 Recent work that should be credited and reused

Phase fourteen already shares derived rules for the two named calculi and proves profile contracts under transitive-container assumptions. Earlier maintained checkpoints normalized source correspondence, separated stable program data from proof dependencies, shared implication derivations and reduced repeated runtime work. Those are completed reductions, not fresh opportunities [2].

The latest reviewed commits also give `EnumDom` an arbitrary-state `halts_iff`, shorten its seed proof, and replace its per-node AX proof by the same checked-word template already used for `EnumPerm`. The proposal queue contains a QHyp-based quotient unification patch, normalized/balanced Rocq source certificates, shared implication cores, scheduler abstractions, decoder-prefix contracts and direct seed-model constructions [4, 5, 6]. The present work builds on that state. In particular, the existing quotient unification and the older `relSat_congr`-through-`sat_quotient` idea should receive their existing attribution when adopted.

2 First reduction: infer successor annotations from the program

2.1 The redundant information

The current AX certificate is a list of pairs

$$[(a_0, n_1), (a_1, n_2), \ldots, (a_{155}, n_{156})].$$

An atom names a previously verified register macro. The second component is the address of the next node. At each step, the current checker compares the literal program node with that pair of macro and successor addresses. But the program already contains the successor. Storing it again makes the certificate longer and creates an extra generated representation to maintain [7].

The proposed certificate is simply

$$[a_0, a_1, \ldots, a_{155}].$$

The checker reads the successor from the literal program. In schematic form:

```
check p n []       = (n == gate)
check p n (a::as)  =
  match p.nodes n with
  | (node m, node next) => (m == a.node) && check p next as
  | _                   => false
```

The resulting statement still concerns the literal program. The data generator does not choose an unchecked successor, and the checker does not substitute an abstract program for the existing one.

2.2 The soundness theorem

Let $\mathrm{Clean}(s)$ express the scratch-register invariant. For each atom a, suppose the established macro contract is

$$\mathrm{Clean}(s) \Longrightarrow \mathrm{Run}_p(a.\,\mathrm{node}, s, a.\,\mathrm{effect}(s)),$$

and that its effect preserves cleanliness. Supply the gate contract at node 457. Then one induction on the atom list proves

$$\mathrm{check}(p, n, w) = \mathrm{true} \ \wedge \ \mathrm{Clean}(s) \Longrightarrow \mathrm{Run}_p(n, s, \mathrm{eval}(w, s)).$$

The empty case uses the gate contract. In the nonempty case the checker exposes the exact two children of the literal node. The first macro contract and the induction hypothesis compose through the existing sequencing theorem. This is the same proof obligation as before, with fewer certificate fields.

2.3 Measured result and adoption scope

The local-only variant changes `L2AxiomsPerm.lean` from 161 lines / 7,563 bytes to 155 lines / 6,533 bytes. The saving is 1,030 bytes (13.62 percent) in that file. There is no additional project module. This is a small change, but its reduction is genuine for K215's own selected source closure.

The archive includes the replacement sources, an applicable patch, the synchronized generator, a replay driver, and typed clients for the exported results. The literal word remains kernel-checked. The producer reproduces both proposed AX files byte for byte. Including the two files and producer, the

local option changes 537 lines / 23,903 bytes to 532 lines / 21,926 bytes. Before production adoption, run the repository's affected-descendant and endpoint audits. The included check establishes the component and its targeted clients; it does not stand in for the physical-site certificate checks.

This pattern is reusable: a certificate should carry semantic choices that the checker cannot infer cheaply. It need not repeat control-flow fields already available in the literal object being checked. Applying that principle to another certificate requires a new soundness argument; it is not a license to omit independent obligations.

3 Second reduction: one AX semantic proof for two programs

3.1 The exact difference between the words

After erasing addresses, the dominated and permuted AX words have 156 atoms each. They agree except at zero-based positions 6 and 7:

Word	Position 6	Position 7
Dominated	`readp1`	`cons`
Permuted	`cons`	`readp1`

The original annotated lists differ in three tuples around that swap, including two successor addresses. The source audit reports the exact tuples. The semantic distinction is not a swap of two decoded fields: it is commuting one parameter read past one formula constructor.

The read updates only $(\texttt{p1}, \texttt{p1})$, using those fields. The constructor updates only $(\texttt{tw}, \texttt{ws})$, using those fields. Thus for every abstract state s,

$$\mathrm{cons}(\mathrm{readp1}(s)) = \mathrm{readp1}(\mathrm{cons}(s)).$$

In the delivered Lean development this equality is `rfl`: both sides reduce to the same record. There is no valid-code hypothesis, zero-register hypothesis, or bounded test set.

The two whole-word evaluations consequently agree for every abstract state. The proof `eval_permuted` is a Boolean case split followed by reflexivity. The large symbolic evaluation of the AX routine is performed once and reused by both compatibility modules. In addition to testing a wrong start node, a truncated word and a wrong first atom, the shared audit rejects each program paired with the other program's word. Equal effects do not erase the separate literal binding.

3.2 The shared interface

The new `AXWord` module contains the atom meanings, cleanliness preservation, generic checked-word theorem, shared word, commutation lemma and symbolic AX evaluation. Each lane supplies its existing transferred macro contracts and its own kernel-checked binding to the literal DAG. The four downstream interfaces remain available: `f236`, `l236`, `c236`, and `ax_eval`.

This is sharing inside the formal development. The current Python producer already shares a template; template sharing alone still leaves two separately maintained formal definitions and two checked mathematical derivations. The common module removes that logical duplication.

3.3 Two different size objectives

Selected boundary	Before	After
AX sources for both programs: lines	322	226
AX sources for both programs: bytes	15,099	9,754
K215 AX source boundary alone: lines	161	195
K215 AX source boundary alone: bytes	7,563	8,511

For the two-lane maintenance boundary, sharing saves 96 lines and 5,345 bytes. For a consumer interested only in the current K215 closure, the shared module plus its Perm adapter is larger by 948 bytes and 34 lines than the current single file. This is why the local annotation-erasure variant is the first recommendation when K215 alone is the size objective. Choose the common core when sharing the two developments is the desired maintenance objective. Do not add the savings from these mutually exclusive patches. Including the synchronized generator, the shared alternative changes 537 lines / 23,903 bytes to 296 lines / 12,546 bytes.

4 Rocq: separate the codec from the command semantics

4.1 Where duplication remains

The maintained Lean `EnumPerm` separates a command datatype, the working registers, `cstep`, `pack`, `decode`, `fetch` and `execAt`. Several Rocq clients still inspect the same numerical command fields independently. The new `CommandFactor.v` checks the Lean-style organization against the current Rocq definitions; this is a cross-language transfer of an existing successful design [8].

Let $\mathrm{fetch}(s) = (z, n)$ and $\mathrm{decode}(z) = (c, r)$. The checked theorem states

$$\mathrm{enum_step}(s) = \mathrm{map}\big(\mathrm{restore}(r, n), \mathrm{xstep}(c, \mathrm{tupof}(s))\big).$$

It is an equality of the actual current definitions, for all natural-valued states. It does not assume that a formula code decodes to a well-formed formula. That unrestricted domain matters because the enumerator intentionally processes arbitrary numerical candidates.

4.2 Two codec laws replace repeated case splits

The adapter proves

$$\mathrm{decode}(\mathrm{enc_fields}(\mathrm{fields}(c), r)) = (c, r) \quad \text{and} \quad \pi_2(\mathrm{decode}(z)) \le z.$$

The first law supports execution of a compiled command list. The second controls the proof-list pointer in candidate draining. Command case analysis is confined to the codec proof and the command semantics; clients can use those laws instead of reopening nested pairing inverses.

For a predicate I on working registers, preservation by every `xstep` lifts to preservation by the encoded enumerator. Likewise, if an `xstep` halting result from I implies P, then an enumerator halting result from I implies P. The concrete stack invariant and falsity conclusion instantiate these general lifting theorems. Their proofs do not appeal to the old enumerator-level invariant theorem, so the apparent simplification is not circular.

Client declaration through its `Qed`	Existing lines	Factored lines
`step_read`	20	10
`step_pl`	22	9
`Inv_step`	24	2
`halt_step`	12	2
Total	78	23

The complete additive adapter is 145 lines / 6,312 bytes. The four smaller client bodies do *not* establish a net 55-line reduction. A production saving must be measured after moving the common command definitions into a shared module and removing the displaced decoder cases. The supplied pilot establishes the equivalence needed to perform that relocation safely.

4.3 Verification and next integration boundary

Eleven modules were freshly compiled with Coq 8.18.0, all seven selected adapter assumption reports are closed, and recursive `coqchk CommandFactor` passed. The driver reads ten bound dependencies from the pinned repository: eight exact K215 sources and two arithmetic support files. The latter were independently verified against the production-pinned K230 reference; they differ only by CRLF-to-LF line endings, with definitions and proofs unchanged. In disposable compatibility copies only, `Stdlib` import prefixes are translated to `Coq`; original and compiled hashes are recorded. This is not a fresh Rocq 9.1.0 production endpoint build.

The next integration should place the command type, working-register tuple, codec, and execution in one module; let soundness, completeness and the L2 root simulation consume it; preserve public aliases where necessary; and regenerate the affected files. The main benefit is that correctness of the field decoder has one owner. This makes the current proof shorter and makes later opcode or field-order changes easier to review.

5 A larger mathematical simplification: direct Infinity

The current lower chain derives a singleton-closed collection, constructs an indexing apparatus, and then obtains ordinary Infinity through `InductiveAttempts`. There is a more direct route: use the seed property on the relation that takes a tagged set to a raw von Neumann successor. The seed's closure clause already permits the reflexive subset. That supplies the successor closure needed by Infinity without building a separate sequence of indices. This section proves the mathematical core and identifies the exact repository adapters [9, 10, 11].

5.1 The seed property

Work with an arbitrary domain V and binary membership relation ε. Write $X \subseteq_\varepsilon Y$ for $\forall w\,(w\ \varepsilon\ X \Rightarrow w\ \varepsilon\ Y)$. The relevant seed property for a relation R and root a is

$$\begin{aligned}\mathrm{Seed}_\varepsilon(R,a)\ \equiv\ \exists C\ [&(\forall y\ R(a,y) \Rightarrow \exists z\ (R(a,z) \wedge z\ \varepsilon\ C))\\ &\wedge\ \forall y\ \varepsilon\ C\ \exists x\ [R(x,y) \wedge\\ &\qquad \forall x' \subseteq_\varepsilon y\ \forall y'\ R(x',y') \Rightarrow \exists z\ (R(x',z) \wedge z\ \varepsilon\ C)]].\end{aligned}$$

The standalone files reproduce this exact shape. No extensionality, foundation, choice, or uniqueness of witnesses is assumed in the next lemma.

Lemma 5.1 (A serial relation already has a closed seed container)**.** *Let $P \subseteq V$, let $P(a)$ hold, and assume*

$$R(x,y) \Rightarrow P(y), \qquad P(x) \Rightarrow \exists y\, R(x,y), \qquad \mathrm{Seed}_\varepsilon(R,a).$$

There is C such that some R-successor of a belongs to C, every member of C satisfies P, and

$$y \,\varepsilon\, C \quad \Longrightarrow \quad \exists z\, (R(y,z) \wedge z \,\varepsilon\, C).$$

Proof. Take a seed witness C. For $y \,\varepsilon\, C$, the seed gives x with $R(x,y)$, hence $P(y)$. Seriality gives a witness to $R(a,-)$, and the seed's first clause places some such witness in C. For closure, take $y \,\varepsilon\, C$, choose a witness to $R(y,-)$ by seriality, and apply the seed's closure clause with $x' = y$. The required inclusion is $y \subseteq_\varepsilon y$, proved by the identity implication. □

This is the conceptual reduction. The seed condition has substantial strength already at the reflexive-subset instance. A proof of successor closure need not use a more elaborate special relation merely to manufacture an index object.

5.2 Tagged raw successors

Fix a marker t with no raw members, and write $T(x)$ for $t \,\varepsilon\, x$. Define

$$\mathrm{Succ}_\varepsilon(x,y) \;\equiv\; \forall z\, [z \,\varepsilon\, y \longleftrightarrow z \,\varepsilon\, x \vee z = x].$$

Thus y is an exact raw successor of x. If $T(x)$ and $\mathrm{Succ}_\varepsilon(x,y)$, then $T(y)$, by inserting $z = t$ in the displayed biconditional.

Take

$$R(x,y) \equiv T(x) \wedge \mathrm{Succ}_\varepsilon(x,y).$$

The existing `raw_aug` theorem, with a true matrix, supplies successors. The existing `raw_singleton` supplies $e = \{t\}$: it is tagged and has no tagged members, since t itself is untagged. The serial-container lemma gives a raw collection C of tagged sets, containing a successor of each member and a successor of e.

Bounded tagged comprehension gives a tagged I whose tagged members are exactly

$$\{e\} \cup \{z : T(z) \wedge z \,\varepsilon\, C\}.$$

The existing raw pair/transitive-bound lemmas provide a bound for that comprehension. If $x = e$, the seed's root clause supplies a successor in C. If $x \,\varepsilon\, C$, the closure lemma supplies one. Thus I is inductive on the tagged domain, with an empty tagged witness e.

There is no equality-free obstacle at this stage. `KLowerSeed.FormSeed` quantifies over arbitrary first-order `Form`, including literal equality; the earlier indiscernibility bridge has already established that interface. With output in slot 0, input in slot 1, and the marker in parameter slot 2, the successor relation can be written as

```
succF := fAll (fIff (fMem 0 1)
                  (fOr (fMem 0 2) (fEq 0 2)))
relF  := fAnd (fMem 2 1) succF
```

The additional binder shifts output/input to slots 1/2 in `succF`. The delivered actual-library adapter checks the precise satisfaction equation and connects this formula to the relation used in the seed theorem.

5.3 Passing to the quotient

Let $q : A \twoheadrightarrow B$ be a surjection. Suppose raw membership r on A and interpreted membership E on B have the presentation

$$E(q(a), q(b)) \quad \longleftrightarrow \quad \exists z\ (r(z, b) \wedge q(a) = q(z)). \tag{$*$}$$

This is exactly the shape of membership obtained by saturating raw membership under the bisimulation quotient. Its establishment in the repository is a separate adapter, using the existing quotient-membership and equality lemmas; that adapter is included and checked here.

Proposition 5.2 (Raw successors descend to genuine successors)**.** *If* $\mathrm{Succ}_r(x, y)$*, then*

$$\forall b \in B \quad E(b, q(y)) \longleftrightarrow E(b, q(x)) \vee b = q(x).$$

Proof. Write $b = q(a)$ using surjectivity. Expand both membership statements using $(*)$. A witness z belonging rawly to y either belongs rawly to x, or equals x. The two cases give the two disjuncts. Conversely, a witness for membership in $q(x)$ also belongs rawly to y; if $q(a) = q(x)$, use the raw witness $z = x$. □

Theorem 5.3 (Infinity descends)**.** *If e has no raw members, $r(e, I)$, and every raw member x of I has an exact raw successor y which is also a member of I, then $q(e)$ is empty and $q(I)$ is inductive under E.*

Proof. An E-member of $q(e)$ would give a raw member of e by $(*)$, a contradiction. The raw witness e shows $E(q(e), q(I))$. For $b \in_E q(I)$, use $(*)$ to find $x \in_r I$ with $b = q(x)$. Choose a raw successor $y \in_r I$. The previous proposition gives the required successor of b, and the witness y places $q(y)$ in $q(I)$. □

5.4 What this can remove, and what remains to be done

The archive contains checked generic seed-closure and quotient-transport results, and their actual-library Lean adapters. Their formal parameters make the integration boundary visible. The production integration should expose a ZF-model constructor taking ordinary Infinity as an explicit input. K215 can then provide that input directly; the existing singleton/index route can remain as a compatibility client for the older general interface.

`InductiveAttempts.lean` is 397 lines / 18,624 bytes and is in K215's current closure. Those numbers are a description of the possible detour, not a measured net saving: another import may keep the module reachable, and the new adapters also occupy source. `FiniteSets` and singleton material are used elsewhere, including the well-founded-part development. Their disappearance must not be assumed. Likewise, the current `QHyp` record contains a singleton field; merely proving direct Infinity does not remove the obligation to build that field. Splitting a smaller base interface or changing the final constructor is necessary before claiming removal of the singleton proof.

5.5 Completed formal checks for the direct route

The delivered Lean development goes beyond an abstract interface sketch. Its three new modules freshly compile with 37 pinned repository modules, giving a **40-module fresh check**. They have the following precise roles:

New module	Lines	Bytes	Result
`DirectInfinity.lean`	141	5,996	General closure and quotient lemmas
`KLowerDirectInfinity.lean`	110	4,087	Actual FormSeed and tagged domain
`KLowerDirectInfinityQuotient.lean`	53	2,325	Actual quotient, ordinary Infinity
Total	304	12,408	

The final `ordinary_quotient_infinity` theorem establishes exactly the existential empty-set and ordinary-successor clause of the current `ZFAxioms.inf` field, on `KQ1Model.quotientMember`. Its assumptions are `FormSeed`, raw irreflexivity, an empty raw marker, a tagged witness, and the current quotient interface. It does not call the old singleton/index proof. The final theorem's printed axioms are precisely `propext`, `Classical.choice`, and `Quot.sound`. The generic seed and tagged induction lemmas have no axioms; the generic quotient transport uses `propext`.

An independent Rocq/Coq core supplies the same seed and ordinary-quotient results. Its 169 lines / 7,881 bytes freshly compile under Coq 8.18.0, all eight assumption reports are closed, and recursive `coqchk -o` reports no axioms. For tagged dependent pairs it uses equality of the underlying representatives explicitly; the quotient map must respect that relation. This avoids assuming proof irrelevance for arbitrary dependent pairs in Rocq. The ordinary quotient results specialize to raw equality and have the same mathematical conclusions as Lean.

The precise remaining step is now *endpoint integration*, not the mathematical construction of Infinity. Extract a `KQ1ZFCore` whose model constructor accepts ordinary Infinity explicitly; retain the existing route as a compatibility wrapper; and supply this checked theorem on the K215 route. The current `QHyp` singleton field still exists. A full endpoint rebuild and a fresh import census must precede any claim that the old dependency has been removed.

6 Validation policy and reproducibility

6.1 The exact boundaries in this packet

1. **Source audit.** A read-only script reproduces the endpoint import closure and hashes the relevant production files. It independently checks the 156-atom lists and their exact two-operation swap.
2. **Lean AX components.** The local and shared alternatives are checked separately: 35 project modules for the local variant and 36 for the shared variant. The build receipts bind the source files and their imported objects; the targeted root/seed and API clients are included. The package reports the selected component scope, not a new 178-module machine endpoint build.
3. **Rocq command adapter.** A fresh compatibility build and recursive kernel check establish the new adapter against exact current definitions. The standard-library namespace rewrite is explicit and occurs only in temporary copies, with both hashes retained.

4. **Direct Infinity.** Generic lemmas are formally checked; the separate status note above identifies how much of the actual repository adapter was compiled. Ordinary Infinity, the complete ZF-model endpoint, and an internally encoded PA/ZFC derivation are different acceptance boundaries.

Every compiler result in the archive is attached to an identified source and toolchain. No numerical simulation is used to certify nonhalting. Finite checks serve only their stated role: binding a literal program or checking a finite certificate that is connected to a universal theorem.

6.2 Reproduction commands

The archive's `README.md` gives exact replay commands and the directory layout. The essential sequence is:

```
git checkout 6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95
python3 tools/audit_snapshot.py /path/to/ProveIt --output audit.json
# Run one AX alternative with its supplied Lean driver.
# Run the Rocq adapter driver against the same checkout.
# Replay the direct-Infinity core using its supplied commands.
pdflatex -halt-on-error report.tex
pdflatex -halt-on-error report.tex
```

Proof tools should be serialized on a constrained machine, as the maintained repository's verification drivers already require. A clean targeted build is preferable to accepting stale descendants because their source files happen to be unchanged. Full endpoint acceptance should follow the repository's existing source-transitive verification procedure.

7 Suggested order of work

First: adopt the local AX erasure if K215 source size is the goal. It is small, has a direct inductive soundness argument, reduces an actual selected source, and preserves the meaningful downstream interfaces. Its producer is already synchronized and its output replayed. Recheck the affected endpoint closure at the chosen final revision.

Second: choose the shared AX core when maintaining both lanes. The all-state commutation proof is unusually simple and removes duplicate symbolic reasoning. Keep its two literal bindings separate. Use the two-lane size boundary when reporting its benefit.

Third: move Rocq's decoder behind one command interface. The additive adapter proves the transition equivalence required for the move. Complete the relocation, rewrite the L2 root against the same interface, then measure the complete changed source boundary. This is a natural way to align the two proof developments without forcing them to use identical tactics.

Fourth: integrate the direct seed-to-Infinity route. The central mathematical observation, actual formula, tagged comprehension and quotient presentation are now formalized: reflexive subset in the seed closure gives a closed collection for any serial relation, and successors give ordinary Infinity. Supply this theorem to the final well-founded-model adapter and check the resulting endpoint before retiring old imports. A closure comparison should then determine the real reduction.

The existing quotient-unification and normalized Rocq certificate proposals remain complementary; adopt them with their recorded provenance and final validation gates.

Artifact and attribution notes

All proof changes in this packet are proposals against the pinned revision. The GitHub repository was not modified. The archive contains the report source and compiled PDF, replacement Lean sources and patches, the Rocq adapter, direct-Infinity prototypes, replay tools, source hashes and compiler receipts. Temporary dependency trees and compiled object caches are excluded.

The machine, existing proof infrastructure, and borrowed definitions remain attributed to ProveIt and the contributors identified in its notices. The Rocq command interface is explicitly adapted from the maintained Lean design. The shared AX refactor extends the repository's existing checked-word method. The direct successor application of the seed property is presented as the new mathematical proposal in this packet, without a claim of literature-wide priority.

References

[1] ProveIt contributors. *Formal correctness of the exact 215-state machine*. Pinned K215 verification record. Repository verification README.

[2] ProveIt contributors. *Fourteenth K215 proof-simplification checkpoint*. Phase-fourteen evidence and source census.

[3] ProveIt contributors. `KEndpoint215.lean`. Exact Lean endpoint and assumptions.

[4] ProveIt commits `ab6ceabe32ee` and `6d9800d1ca67`. Checked AX word for the dominated program; Arbitrary-state EnumDom halting theorem.

[5] ProveIt contributors. *K215 proof-simplification proposals*. Proposal index; some introductory checkpoint descriptions in the index are historical.

[6] ProveIt contributors. *One quotient development through the QHyp interface*. Attributed quotient-unification proposal.

[7] ProveIt contributors. `L2AxiomsPerm.lean` and `L2AxiomsDom.lean`. Pinned Perm AX word; Pinned Dom AX word.

[8] ProveIt contributors. `EnumPerm.lean` and current `K215F` Rocq enumerator modules. Lean command/codec organization; Rocq K215 modules.

[9] ProveIt contributors. `KLowerSeed.lean`, `KLowerRaw.lean`, `KLowerTag.lean`. FormSeed with literal equality; Bounded tagged comprehension.

[10] ProveIt contributors. `KLowerQHyp.lean` and `KQ1ZF.lean`. Current tagged quotient interface; Current ZF-model endpoint.

[11] ProveIt contributors. `InductiveAttempts.lean`. Existing singleton-index-to-Infinity development.